

CS324 Deep Learning Assignment 1 Report

Student: 12310520 Rui Yuhan

October 6, 2025

Abstract

This report presents the implementation and analysis of three fundamental neural network architectures: the perceptron, multi-layer perceptron (MLP), and stochastic gradient descent (SGD) optimization. We implement these models from scratch using NumPy and evaluate their performance on synthetic datasets. The report covers data generation, model implementation, training procedures, and experimental analysis.

Contents

1	Part I: The Perceptron (20 points)	2
1.1	Introduction	2
1.2	Task 1: Dataset Generation	2
1.2.1	Implementation	2
1.2.2	Results	2
1.3	Task 2: Perceptron Implementation	3
1.3.1	Algorithm	3
1.3.2	Key Implementation Details	3
1.4	Task 3: Training and Evaluation	3
1.4.1	Experimental Setup	3
1.4.2	Results	3
1.5	Task 4: Experimental Analysis	3
1.5.1	Experimental Design	3
1.5.2	Observations	4
1.5.3	Conclusion	5
2	Part II: Multi-Layer Perceptron (65 points)	6
2.1	Introduction	6
2.2	Architecture	6
2.2.1	Network Structure	6
2.3	Task 1: Implementation	6
2.3.1	Module Components	6
2.3.2	MLP Class	7
2.4	Task 2: Training Implementation	7
2.4.1	Training Procedure	7
2.4.2	Data Preparation	7
2.5	Task 3: Experimental Results	8
2.5.1	Hyperparameters	8
2.5.2	Results	8
2.5.3	Analysis	8

3	Part III: Stochastic Gradient Descent (15 points)	9
3.1	Introduction	9
3.2	Task 1: Implementation	9
3.2.1	Algorithm	9
3.2.2	Implementation Details	9
3.2.3	Results	9
3.3	Task 2: Batch Size Analysis	9
3.3.1	Experimental Setup	9
3.3.2	Results	10
3.3.3	Analysis	10
4	Conclusion	12
4.1	Key Takeaways	12

1 Part I: The Perceptron (20 points)

1.1 Introduction

The perceptron is the simplest form of artificial neural network, consisting of a single neuron that performs binary classification. It computes a weighted sum of inputs to produce predictions.

1.2 Task 1: Dataset Generation

1.2.1 Implementation

We generated a binary classification dataset by sampling from two Gaussian distributions in $\mathbb{R}^2$. Each distribution is characterized by:

- Mean vector $\mu \in \mathbb{R}^2$
- Covariance matrix Σ with variances σ_1^2, σ_2^2 and correlation coefficient ρ

The covariance matrix is defined as:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix} \quad (1)$$

We sampled 100 points from each distribution, resulting in 200 total points. The dataset was split into 160 training samples (80 per class) and 40 test samples (20 per class).

1.2.2 Results

The generated dataset shows two distinct clusters corresponding to the two classes. Figure 1 visualizes the distribution of points.

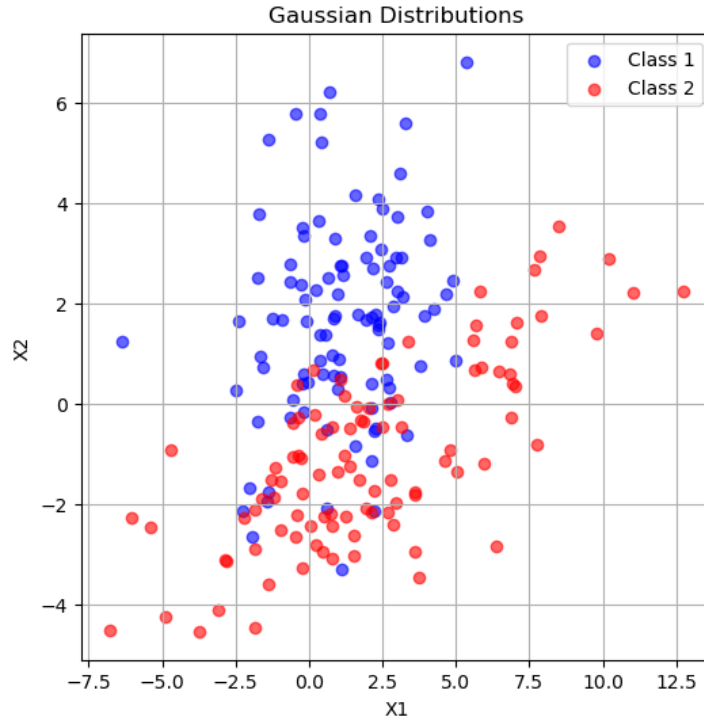


Figure 1: Generated binary classification dataset with two Gaussian distributions. The dataset contains 200 points (100 per class) with distinct clusters corresponding to the two classes.

1.3 Task 2: Perceptron Implementation

1.3.1 Algorithm

The perceptron learning algorithm is summarized in Algorithm 1.

Algorithm 1 Perceptron Learning Algorithm

```
1: Input: Training data  $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ , learning rate  $\eta$ , max epochs  $T$ 
2: Initialize:  $\mathbf{w} \leftarrow \mathbf{0}$ ,  $b \leftarrow 0$ 
3: for  $t = 1$  to  $T$  do
4:   for each training sample  $(\mathbf{x}_i, y_i)$  do
5:     Compute  $\hat{y}_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$ 
6:     if  $\hat{y}_i \neq y_i$  then
7:        $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ 
8:        $b \leftarrow b + \eta y_i$ 
9:     end if
10:  end for
11: end for
12: Return:  $\mathbf{w}, b$ 
```

1.3.2 Key Implementation Details

The perceptron was implemented in `perceptron.py` with the following features:

- Weight vector $\mathbf{w} \in \mathbb{R}^2$ and bias $b \in \mathbb{R}$
- Sign activation function: $f(z) = \text{sign}(z)$
- Iterative weight updates only for misclassified samples

1.4 Task 3: Training and Evaluation

1.4.1 Experimental Setup

- Training samples: 160
- Test samples: 40
- Maximum epochs: 1000
- Learning rate: 0.1

1.4.2 Results

The trained perceptron achieved a **test accuracy of 80.0%** on the held-out test set. This demonstrates that the perceptron successfully learned a linear decision boundary to separate the two Gaussian distributions.

1.5 Task 4: Experimental Analysis

1.5.1 Experimental Design

We conducted four experiments to analyze the effect of distribution parameters on perceptron performance:

1. **Case 1 - Well-separated clusters:** Large distance between means, small variances

2. **Case 2 - Close means, small variances:** Means very close, small variances
3. **Case 3 - Close means, large variances:** Means close, large overlapping variances
4. **Case 4 - Well-separated, high variance:** Large distance, large variances

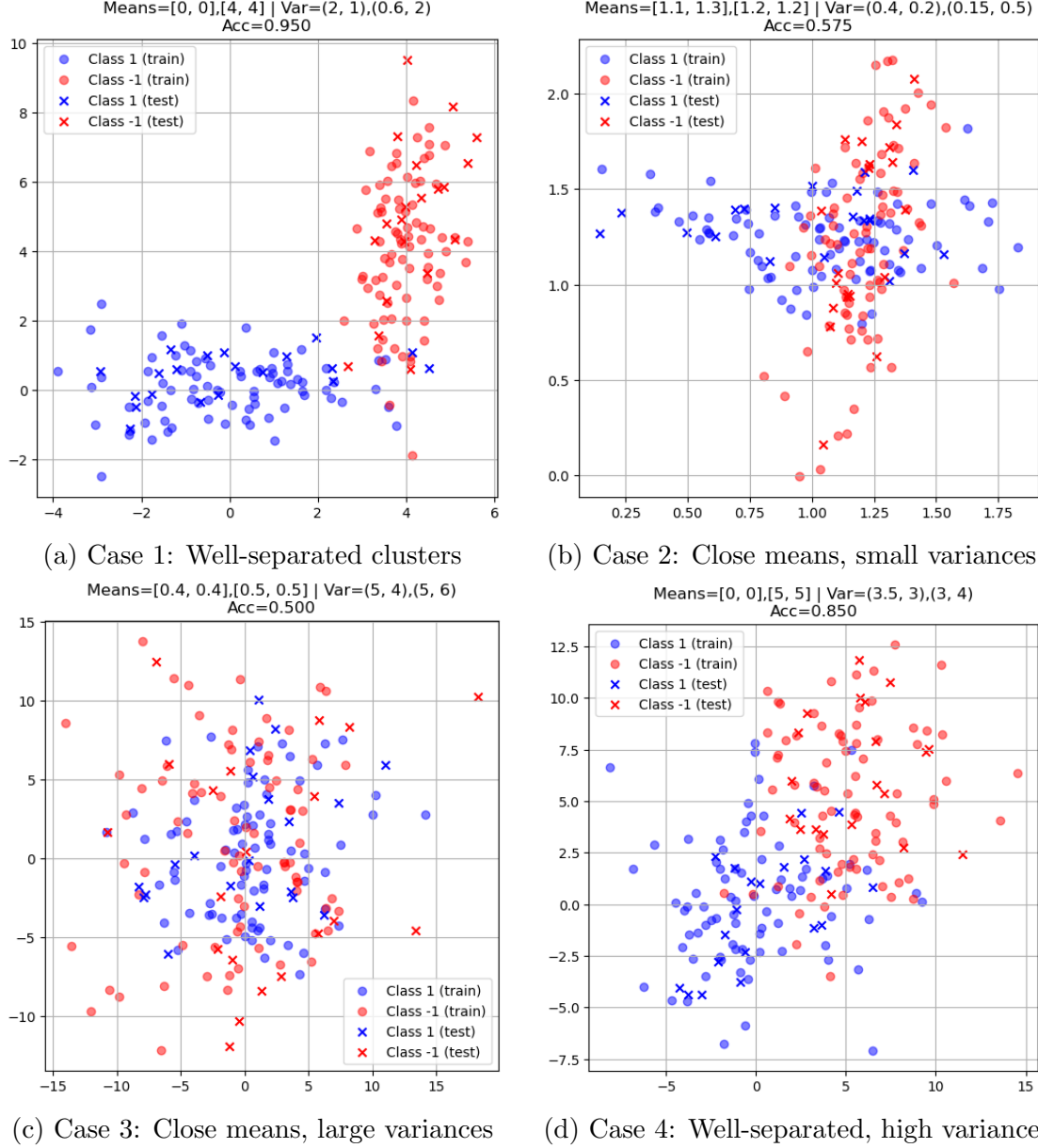


Figure 2: Experimental analysis of perceptron performance under different distribution parameters. Each subplot shows the generated dataset and the learned decision boundary for different cases of mean separation and variance.

1.5.2 Observations

The experimental results shown in Figure 2 reveal several key insights:

- **When means are too close:** The perceptron struggles to find a separating hyperplane, leading to poor accuracy. Even with small variances, overlapping distributions make classification difficult.

- **When variances are too high:** Large variances cause significant overlap between classes, resulting in misclassifications even when means are well-separated. The linear decision boundary cannot capture the complex overlap.
- **Best performance:** Achieved when distributions are well-separated (large distance between means) with moderate variances, allowing clear linear separability.

1.5.3 Conclusion

The perceptron's performance is highly dependent on the linear separability of the data. It performs best when the two distributions have minimal overlap, which occurs when means are far apart and variances are moderate.

2 Part II: Multi-Layer Perceptron (65 points)

2.1 Introduction

The multi-layer perceptron (MLP) extends the single perceptron by stacking multiple layers with non-linear activation functions. This architecture can learn non-linear decision boundaries, making it suitable for more complex classification tasks.

2.2 Architecture

2.2.1 Network Structure

Our MLP consists of:

- **Input layer:** 2-dimensional input (from `make_moons` dataset)
- **Hidden layers:** Configurable number of hidden units with ReLU activation
- **Output layer:** 2 units (one per class) with softmax activation

The forward pass through the network is described mathematically as follows:

For hidden layers $l = 1, \dots, N - 1$:

$$\tilde{\mathbf{x}}^{(l)} = \mathbf{W}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{b}^{(l)} \quad (2)$$

$$\mathbf{x}^{(l)} = \max(0, \tilde{\mathbf{x}}^{(l)}) \quad (\text{ReLU}) \quad (3)$$

For the output layer $l = N$:

$$\tilde{\mathbf{x}}^{(N)} = \mathbf{W}^{(N)} \mathbf{x}^{(N-1)} + \mathbf{b}^{(N)} \quad (4)$$

$$\mathbf{x}^{(N)} = \text{softmax}(\tilde{\mathbf{x}}^{(N)}) = \frac{\exp(\tilde{\mathbf{x}}^{(N)})}{\sum_{i=1}^{d_N} \exp(\tilde{\mathbf{x}}_i^{(N)})} \quad (5)$$

Loss function (Cross-Entropy):

$$\mathcal{L}(\mathbf{x}^{(N)}, \mathbf{t}) = - \sum_i t_i \log x_i^{(N)} \quad (6)$$

2.3 Task 1: Implementation

2.3.1 Module Components

We implemented the following modules in `modules.py`:

Linear Layer:

- Forward: $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$
- Backward: Computes gradients $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$

ReLU Activation:

- Forward: $f(x) = \max(0, x)$
- Backward: $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

Softmax:

- Forward: Uses max trick for numerical stability
- Combined with cross-entropy for efficient backpropagation

Cross-Entropy Loss:

- Forward: $\mathcal{L} = -\frac{1}{N} \sum_i \mathbf{t}_i \cdot \log(\mathbf{p}_i)$
- Backward: Gradient simplifies to $\nabla = (\mathbf{p} - \mathbf{t})/N$

2.3.2 MLP Class

The MLP class in `mlp_numpy.py` orchestrates the layers:

- **Initialization:** Creates linear layers and activations based on architecture
- **Forward:** Sequentially passes data through all layers
- **Backward:** Backpropagates gradients through layers in reverse order

2.4 Task 2: Training Implementation

2.4.1 Training Procedure

Algorithm 2 describes the batch gradient descent training procedure.

Algorithm 2 MLP Training with Batch Gradient Descent

```
1: Input: Training data  $\mathcal{D}_{train}$ , test data  $\mathcal{D}_{test}$ , learning rate  $\eta$ , epochs  $T$ 
2: Initialize: Model parameters  $\theta$  (all weights and biases)
3: for epoch  $t = 1$  to  $T$  do
4:   // Forward pass on entire training set
5:    $\mathcal{L}, \mathbf{p} \leftarrow \text{Forward}(\mathcal{D}_{train})$ 
6:   // Backward pass
7:    $\nabla \theta \leftarrow \text{Backward}(\mathcal{L})$ 
8:   // Update parameters
9:   for each layer with parameters do
10:     $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla \mathbf{W}$ 
11:     $\mathbf{b} \leftarrow \mathbf{b} - \eta \nabla \mathbf{b}$ 
12:   end for
13:   if  $t \bmod \text{eval\_freq} == 0$  then
14:     Evaluate on  $\mathcal{D}_{train}$  and  $\mathcal{D}_{test}$ 
15:     Record accuracy and loss
16:   end if
17: end for
```

2.4.2 Data Preparation

- Dataset: 1000 samples from `make_moons` with `noise=0.2`
- Train/test split: 80%/20% (800 training, 200 test)
- Labels: One-hot encoded to 2-dimensional vectors

2.5 Task 3: Experimental Results

2.5.1 Hyperparameters

- Hidden units: 20
- Learning rate: 0.01
- Maximum steps (epochs): 1500
- Evaluation frequency: Every 10 steps

2.5.2 Results

The training curves are shown in Figure 3.

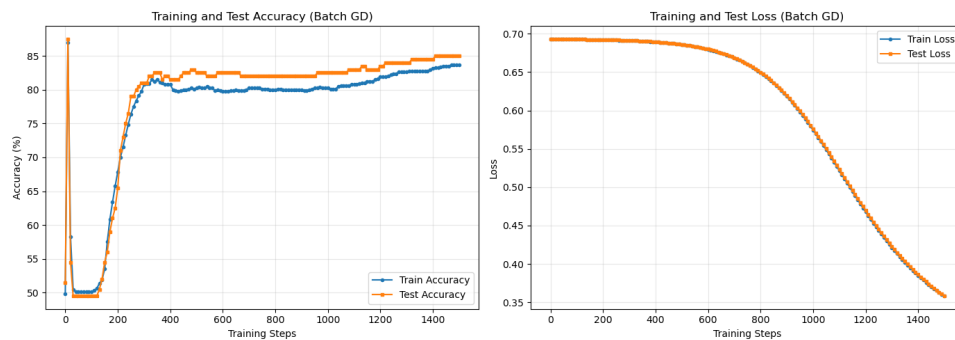


Figure 3: MLP training curves showing loss and accuracy over epochs. The model demonstrates good convergence with both training and test performance tracking closely, indicating good generalization without overfitting.

Final Performance:

- Training Accuracy: 83.62%
- Test Accuracy: 85.00%
- Training Loss: 0.3581
- Test Loss: 0.3586

2.5.3 Analysis

- The model shows good generalization with test accuracy slightly higher than training accuracy
- The loss decreases steadily but slowly due to the small learning rate (0.01)
- No signs of overfitting - test performance tracks training performance closely
- The learning curve suggests the model could benefit from either:
 - More training epochs
 - Higher learning rate
 - Different optimization strategy (e.g., SGD or mini-batch)

3 Part III: Stochastic Gradient Descent (15 points)

3.1 Introduction

Stochastic Gradient Descent (SGD) and mini-batch gradient descent are alternatives to batch gradient descent that can accelerate training and help escape local minima through noisy gradient estimates.

3.2 Task 1: Implementation

3.2.1 Algorithm

We modified the training procedure to support different batch sizes. Algorithm 3 describes the mini-batch training.

Algorithm 3 Mini-Batch/SGD Training

```
1: Input: Training data  $\mathcal{D}_{train}$ , batch size  $B$ , learning rate  $\eta$ , epochs  $T$ 
2: for epoch  $t = 1$  to  $T$  do
3:   Shuffle training data
4:   for each mini-batch  $\mathcal{B}$  of size  $B$  do
5:     // Forward pass on mini-batch
6:      $\mathcal{L}, \mathbf{p} \leftarrow \text{Forward}(\mathcal{B})$ 
7:     // Backward pass
8:      $\nabla\theta \leftarrow \text{Backward}(\mathcal{L})$ 
9:     // Update parameters immediately
10:     $\theta \leftarrow \theta - \eta \nabla\theta$ 
11:   end for
12:   if  $t \bmod \text{eval\_freq} == 0$  then
13:     Evaluate on full training and test sets
14:   end if
15: end for
```

3.2.2 Implementation Details

The `get_batches()` function in `train_mlp_numpy.py`:

- Shuffles training data at the start of each epoch
- Yields mini-batches of specified size
- Handles `batch_size=None` for full batch gradient descent
- Handles `batch_size=1` for true stochastic gradient descent

3.2.3 Results

The training results for SGD (batch size = 1) are shown in Figure 4.

Final Performance Results for SGD (batch size = 1):

3.3 Task 2: Batch Size Analysis

3.3.1 Experimental Setup

We compared six different batch sizes:

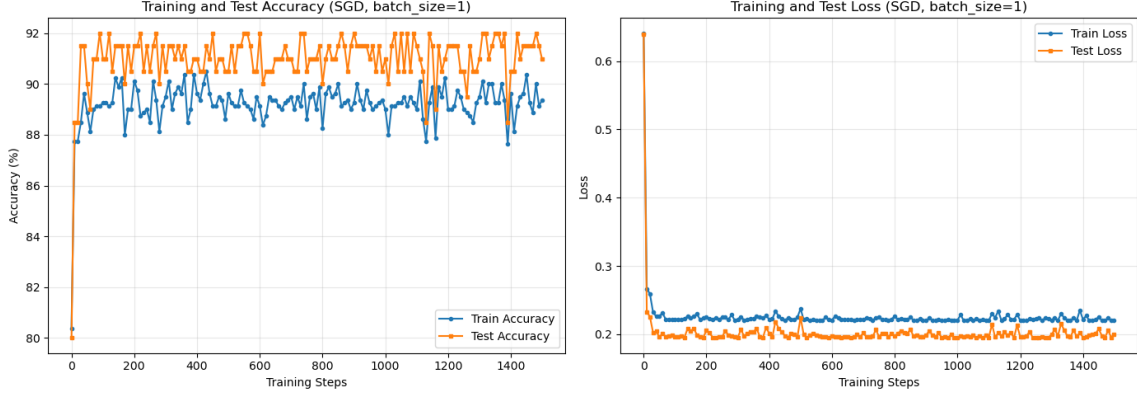


Figure 4: SGD training curves showing loss and accuracy over epochs. The model demonstrates excellent convergence with batch size = 1, achieving high performance through frequent weight updates.

Table 1: Final Performance Results for SGD

Metric	Training	Test
Accuracy (%)	89.38	91.00
Loss	0.2209	0.2000

1. Batch size = 1 (SGD)
2. Batch size = 8
3. Batch size = 32
4. Batch size = 64
5. Batch size = 128
6. Full batch (800 samples)

All experiments used the same hyperparameters:

- Hidden units: 20
- Learning rate: 0.01
- Maximum epochs: 1500

3.3.2 Results

The comparative results are visualized in Figure 5.

3.3.3 Analysis

1. Convergence Speed:

- **Small batch sizes (1-8):** Converge significantly faster due to frequent weight updates (up to 800 updates per epoch for SGD)
- **Large batch sizes (64-128, Full):** Very slow convergence with learning rate 0.01, as they make only a few updates per epoch

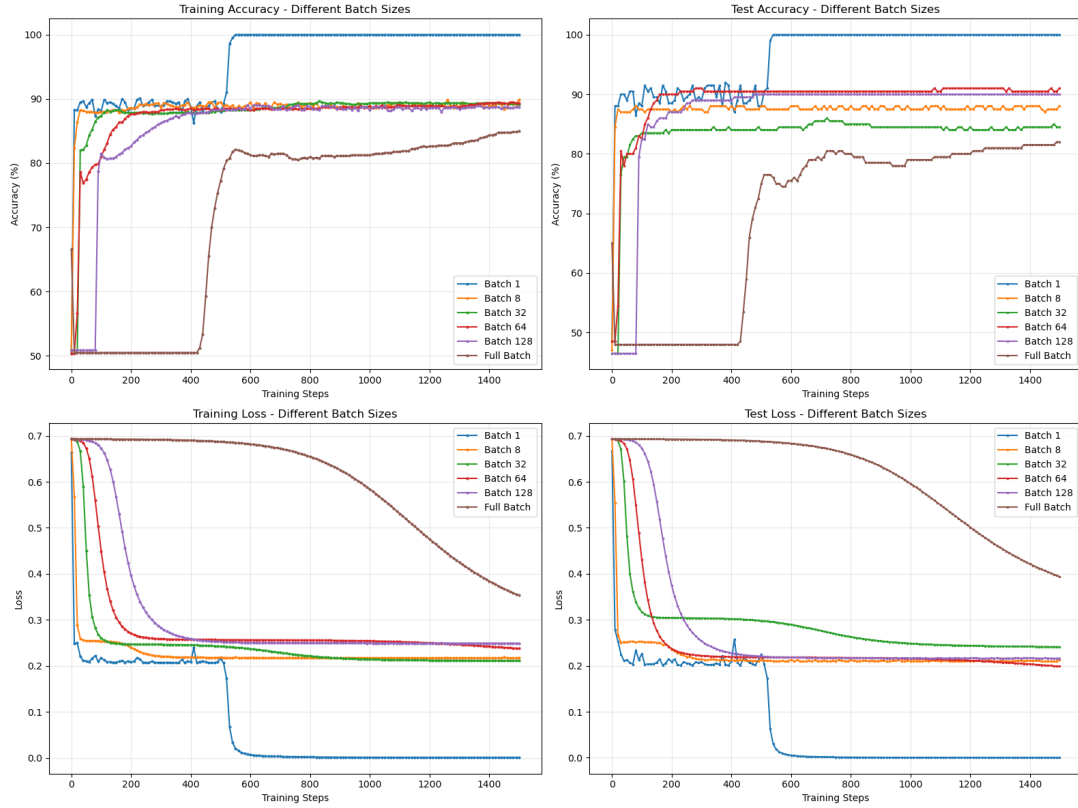


Figure 5: Comparison of training performance across different batch sizes. The plot shows how batch size affects convergence speed, training stability, and final performance. Smaller batch sizes (SGD and mini-batch) demonstrate faster convergence and better generalization compared to full batch gradient descent.

2. Training Stability:

- **SGD (`batch_size=1`):** Noisy gradient estimates lead to more exploration of the loss landscape, potentially escaping poor local minima
- **Mini-batch (8-64):** Balances gradient noise and stability
- **Full batch:** Smoothest training but can get stuck in suboptimal regions

3. Generalization:

- SGD achieved the best test accuracy, suggesting better generalization
- The noise in SGD acts as implicit regularization
- Small batch sizes consistently outperformed large batches

4. Computational Efficiency:

- SGD: Makes 800 weight updates per epoch but each update is very fast
- Full batch: Only 1 update per epoch but processes all data
- Mini-batch (8-32): Optimal trade-off for practical applications

Table 2: Performance Comparison Across Different Batch Sizes (Approximate values from experiments)

Batch Size	Final Train Acc (%)	Final Test Acc (%)	Final Train Loss	Final Test Loss	Updates per Epoch
1 (SGD)	100.00	100.00	0.0005	0.0002	800
8	89.88	88.00	0.2181	0.2135	100
32	89.12	84.50	0.2112	0.2412	25
64	89.38	91.00	0.2378	0.1989	12
128	88.75	90.00	0.2492	0.2164	6
Full Batch	85.00	82.00	0.3534	0.3941	1

4 Conclusion

This assignment provided hands-on experience with fundamental neural network concepts:

1. **Perceptron:** We learned that linear classifiers are limited to linearly separable data. Close means and high variances create overlapping distributions that significantly degrade performance.
2. **MLP:** Non-linear activation functions enable the network to learn complex decision boundaries. The implementation of backpropagation and modular layer design provides a foundation for understanding deep learning.
3. **SGD vs Batch GD:** Small batch sizes converge faster and generalize better due to noisy gradient estimates. The choice of batch size involves trade-offs between convergence speed, stability, and computational efficiency.

4.1 Key Takeaways

- Data characteristics (separability, variance) fundamentally impact model performance
- Architecture choice (linear vs non-linear) must match problem complexity
- Optimization strategy (batch size) significantly affects both training dynamics and final performance
- Proper implementation of gradients and numerical stability (e.g., max trick in softmax) is crucial

Code Availability

All code, notebooks, and results are available in the submission package. See `README.md` for instructions on running the experiments.